Refactorización del script en funciones reutilizables (Bloque 6) / De un script monolítico a funciones reutilizables

✓ **Hecho:** Ver

📘 **UNIDAD U1.3** · Python aplicado para integración con IA

De un script monolítico a funciones reutilizables

En el Bloque 5 construiste una primera versión de tu módulo, y funciona: arma una petición, la envía, procesa la respuesta y devuelve un dato. Pero **"funciona"** y **"está bien organizada"** no son lo mismo, y este bloque se dedica exactamente a esa diferencia.

> No vas a agregarle ninguna función nueva a lo que ya construiste, ni tampoco vas a hacer que resuelva algo que hoy no resuelve. Vas a reorganizar lo que ya existe, para que quede en mejores condiciones antes de blindarlo contra errores en el **Bloque 7**.

Esta lección no requiere instalar nada nuevo ni tocar tu archivo "**.env**": vas a reorganizar el mismo módulo que ya construiste y probaste en el Bloque 5, con las mismas dependencias y la misma credencial que ya tenías configuradas.

🔄 Qué significa exactamente "refactorizar"

Refactorizar tiene un significado preciso, distinto de "mejorar" en general: es cambiar la estructura interna de un código sin cambiar su comportamiento externo.

Si le das a tu función el mismo dato de entrada antes y después de refactorizarla, el resultado que devuelve tiene que ser exactamente el mismo.



Lo único que cambia es cómo está organizado el código por dentro: cuántas funciones hay, qué hace cada una, y cómo se llaman entre sí. Si al reorganizar tu código el resultado cambia, eso no es una refactorización, es un error nuevo, y hay que tratarlo como tal, no como un efecto esperado del proceso.

🧱 Qué tiene de "monolítico" tu primera versión

La función que construiste en el **Bloque 5** hace, en un solo bloque de código, varias cosas a la vez: arma la dirección del servicio, envía la petición, convierte la respuesta a una estructura de **Python**, y extrae el dato específico que te interesa.

Escribirlo así, todo junto, fue lo correcto para esa primera versión: el objetivo era que funcionara de principio a fin, lo más simple posible, sin distraerte todavía con cómo se organiza.

> ☝️ **No es un error que tengas que corregir por haberlo hecho mal**; es, sencillamente, la forma normal

en la que suele nacer un primer intento que funciona. El problema aparece después, cuando ese código necesita crecer o cambiar.

Esa mezcla de responsabilidades en un mismo lugar es justamente lo que hace que una función sea difícil de leer de un vistazo, difícil de probar por partes, y difícil de reutilizar si mañana necesitas, por ejemplo, la misma lógica de conversión de **JSON** pero para una petición distinta.

💡 Un ejemplo concreto de por qué esto importa

Imagina que, más adelante, necesitas usar exactamente la misma lógica que ya escribiste para procesar una respuesta **JSON**, pero ahora sobre los datos que devuelve un servicio distinto.

Si esa lógica está mezclada dentro de una función más grande que también arma la **URL** y envía la petición, no puedes tomar solo esa parte: tendrías que copiar y pegar fragmentos de código, y arrastrar con ellos el riesgo de que la copia se desactualice apenas cambies el original.

Con las responsabilidades separadas en funciones distintas, en cambio, esa reutilización es directa: llamas a la función de procesamiento con los datos que necesites, sin tocar ni una línea del resto.

> 🎯 Ese es, en el fondo, **el beneficio real de refactorizar**: no es una cuestión de estética del código, es lo que determina qué tan fácil o difícil va a ser trabajar sobre él la próxima vez.

🧩 Cómo se reconoce dónde partir una función

El criterio no es partir el código en pedazos parejos, es identificar responsabilidades distintas que hoy están mezcladas en un mismo lugar. En el módulo que construiste, esas responsabilidades suelen separarse en algo como esto:



Una función que arma la URL o los datos de la petición, a partir de los parámetros que recibe.



Una función que envía la petición y devuelve la respuesta cruda, sin preocuparse todavía de



Una función que procesa esa respuesta y extrae el dato específico, sin preocuparse de cómo se

qué hay dentro de esa respuesta.

obtuvo esa respuesta en primer lugar.

Cada una de estas piezas, separada, se puede leer, probar y reutilizar por su cuenta. Y cada una responde a la misma pregunta que ya viste en el Bloque 5, aplicada ahora en la práctica: si para describir qué hace una función necesitas la palabra "y" varias veces, ahí probablemente hay más de una función escondida.

Un ejemplo ilustrativo ayuda a verlo concretamente. Tu función del **Bloque 5**, monolítica, hace todo junto:

```
def obtener_dato_de_prueba(pregunta):
    api_key = os.getenv("GEMINI_API_KEY")
    url = f"https://generativelanguage.googleapis.com/v1beta/models/{modelo}:generateConte
    body = {"contents": [{"parts": [{"text": pregunta}]}]}
    respuesta = requests.post(url, json=body)
    datos = respuesta.json()
    return datos["candidates"][0]["content"]["parts"][0]["text"]
```

Refactorizada, la misma lógica queda repartida en funciones con una sola responsabilidad cada una, y la función original pasa a coordinarlas:

```
def armar_peticion(pregunta):
    api_key = os.getenv("GEMINI_API_KEY")
    url = f"https://generativelanguage.googleapis.com/v1beta/models/{modelo}:generateConte
    body = {"contents": [{"parts": [{"text": pregunta}]}]}
    return url, body

def enviar_peticion(url, body):
    respuesta = requests.post(url, json=body)
    return respuesta.json()

def extraer_texto(datos):
    return datos["candidates"][0]["content"]["parts"][0]["text"]

def obtener_dato_de_prueba(pregunta):
    url, body = armar_peticion(pregunta)
    datos = enviar_peticion(url, body)
    return extraer_texto(datos)
```

Fíjate en algo importante: "obtener_dato_de_prueba" sigue existiendo, con el mismo nombre y el

☝️ **Fíjate en algo importante**: "obtener_dato_de_prueba" sigue existiendo, con el mismo nombre y el mismo parámetro. Quien ya la esté usando en otra parte de tu código no nota ningún cambio; por dentro, ahora delega el trabajo en tres funciones más pequeñas en vez de hacerlo todo ella misma.

Puedes confirmarlo tú mismo con el mismo caso de prueba que usaste al construir tu primera versión: la llamada no cambia

```
print(obtener_dato_de_prueba("Confirma en una frase corta que esta conexión funciona"))
```

Lo que importa comparar aquí no es el texto exacto de la respuesta (eso varía solo, porque el modelo genera contenido nuevo en cada ejecución), sino que ambas versiones, la de antes y la de después de refactorizar, sigan devolviendo una respuesta válida, sin errores nuevos, para la misma pregunta.

🏷️ Los nombres también son parte del diseño

Separar responsabilidades en funciones distintas no sirve de mucho si después esas funciones se llaman de forma genérica, como **"procesar"** o **"hacer_cosa"**.

El nombre de cada función debería dejar claro, sin necesidad de leer su código, qué hace exactamente: algo como **"obtener_respuesta_del_servicio"** o **"extraer_dato_de_respuesta"** comunica mucho más que un nombre corto pero vago.

➡️ Esto no es un detalle menor ni una cuestión de gusto personal: nombrar bien tus funciones es, junto con la responsabilidad única que ya viste, otro de los criterios que la rúbrica de calidad de código usa para evaluar tu módulo, y vas a revisarla con detalle más adelante en esta unidad, en el **Bloque 7**.

🔒 Qué no cambia cuando refactorizas

El resultado final que le llega a quien use tu módulo tiene que seguir siendo idéntico: mismo dato de entrada, mismo dato de salida. Nada de lo que hagas en este bloque debería alterar eso.

Por eso conviene probar tu función con el mismo caso que ya usaste en el **Bloque 5**, antes de empezar a mover código, y volver a probarla con ese mismo caso apenas termines. Si el resultado coincide en ambos momentos, la refactorización salió bien; si no coincide, algo se rompió en el camino y hay que revisarlo antes de seguir.

> 🔄 Este **hábito de comparar el antes y el después** es, en el fondo, la misma idea detrás de uno de los criterios de la rúbrica de calidad de código: un código bien organizado no es el que se ve más corto o más elegante, es el que sigue haciendo exactamente lo que tiene que hacer, de una forma

más fácil de leer y de mantener.

> **Si ves un error**: las causas más comunes aquí son las mismas que ya viste al construir tu primera versión, en "Construye tu módulo Python de consumo de servicios" (una credencial ausente o mal escrita, que termina en un "KeyError: 'candidates'" al leer la respuesta). El código que arma la petición y lee la respuesta no cambió, solo se reorganizó en funciones separadas, así que la causa y la solución son las mismas: revisa tu ".env" antes que tu código.



Siguiente lección

Bloque 6 · Refactorización del módulo

Con esta idea clara (qué es refactorizar, qué responsabilidades separar, cómo nombrarlas, y qué tiene que seguir igual), la siguiente lección de este mismo bloque es donde aplicas esto directamente sobre tu propio módulo, dentro de tu repositorio.

Última modificación: jueves, 24 de septiembre de 2026, 15:02